

Universidad Nacional Autónoma de México.



Facultad de Ingeniería.

Sistemas Distribuidos.

Profesor: Ing. Óscar Manuel Ruíz Hurtado.

Semestre 2027-1

Laboratorio S9.

Mexi Banco: que el aviso no tumbe la transferencia.

Grupo: 2

Alumno:

Flores Herrera Oswaldo

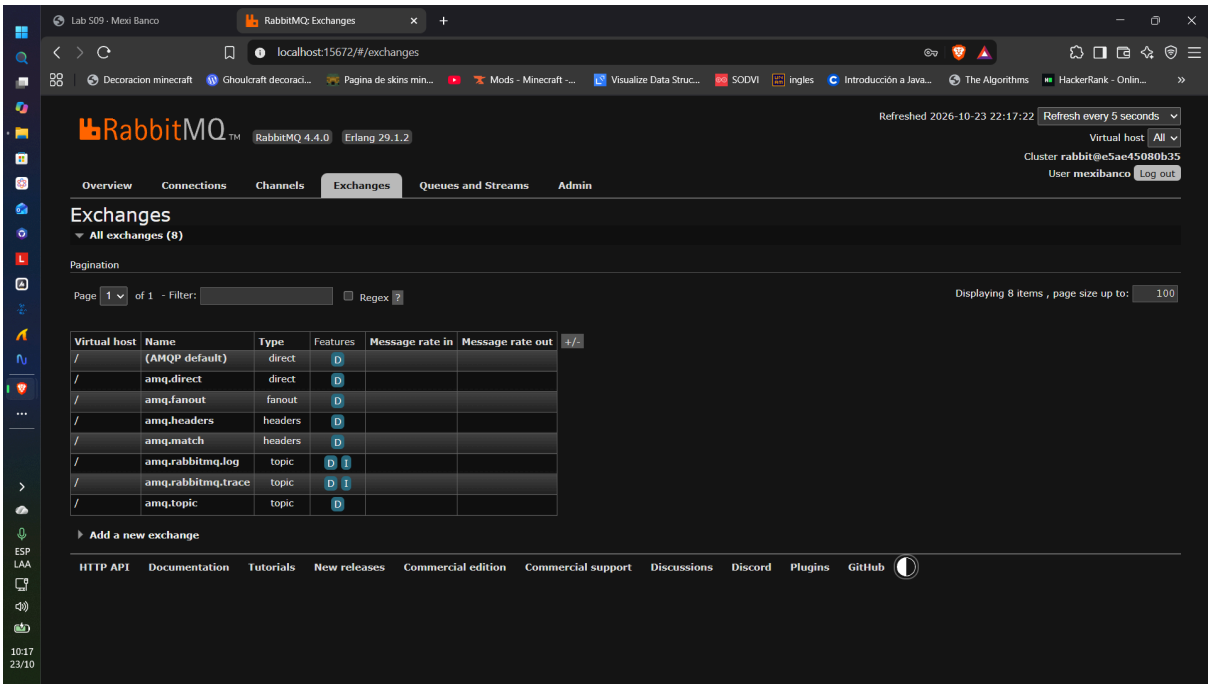
Cd. Universitaria a 24 de octubre de 2026

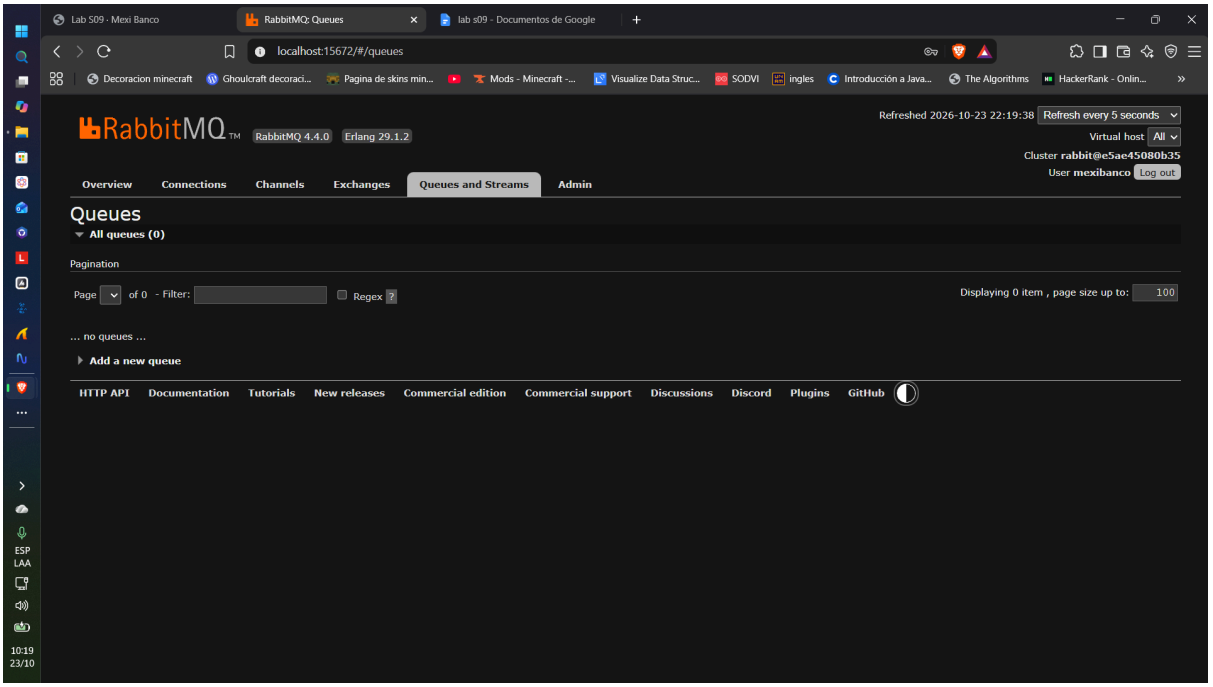
Flores Herrera Oswaldo
Universidad Nacional Autónoma de México

1. El problema

Con notificación apagada, hice una transferencia de 50 de Ana a Beto. Respondió con error, pero la consulta por ABONADA la muestra y los saldos ya se habían movido: Ana quedó con 50 menos y Beto con 50 más.

¿Por qué el cliente recibe un error? Porque la saga esperaba la respuesta HTTP de notificación después de que el dinero ya había llegado a destino; al no contestar, la excepción salió antes de marcarCompletada() y la transferencia se quedó en ABONADA.





Flores Herrera Oswaldo
Universidad Nacional Autónoma de México

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 04:22:19 GMT

{"timestamp":"2026-10-24T04:22:19.686+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"
}
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
[{"id":37,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50.00,"fecha":"2026-10-24
T04:22:19.456490Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actu
alizada":"2026-10-24T04:22:19.527318z"}]
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s http://localhost:8080/api/cuentas/0021800000000000001
{"id":6,"clabe":"0021800000000000001","titular":"Ana","saldo":950.00}
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s http://localhost:8080/api/cuentas/0021800000000000002
{"id":7,"clabe":"0021800000000000002","titular":"Beto","saldo":550.00}
```

2. Publicar.

Transferencia pública un hecho ("esta transferencia se realizó") en un exchange de RabbitMQ y sigue con lo suyo. Lo que hice en el servicio transferencia.

1. eventoId desde que nace la transferencia. Agregué a la entidad Transferencia una columna eventoId (UUID , única) que se genera en el constructor, más un @PostLoad para filas viejas de la S08 que no lo traen. Nace con la fila y se guarda con ella para que, si el evento se publica dos veces, lleve el mismo id; si se generara al publicar, cada reintento parecería un evento nuevo. Es la llave de idempotencia que usó después en la sección 4.

2. El record EventoTransferencia . Es el contrato con quien escucha, por eso es un record propio y no la entidad: si cambio una columna, no se rompe nadie afuera. Es un hecho en pasado, no una orden. Tiene dos fábricas, realizada y revertida , y routingKey() devuelve transferencia.realizada o transferencia.revertida según
el tipo.

3. Configuración (ConfiguracionMensajeria). Declara el exchange transferencias , de tipo topic y durable (sobrevive a un reinicio del broker), y un Jackson2JsonMessageConverter con el ObjectMapper de Spring Boot para que los eventos viajen como JSON y la fecha salga en ISO-8601. Transferencia sólo conoce el exchange; no sabe qué colas hay detrás, esas son de quien consume.

4. Publisher confirms (application.ym). Configuré publisher-confirm-type. correlated , publisher-returns: true , template.mandatory: true y connection- timeout: 2s . Sin confirmación, publicar regresa en cuanto los bytes salen por el socket; si el broker se cae en ese instante, el evento se pierde y nadie se entera.

5. PublicadorDeEventos . Pública el evento con entrega persistente (se escribe a disco) y el eventoId como messageId , usando una CorrelationData para unir la confirmación (que llega después, por otro hilo) con ese mensaje. Luego espera la confirmación hasta espera-confirmacion-ms (5 s). Lanza ventoNoPublicadoException si no hay conexión, si no llega confirmación a tiempo, si el broker responde nack, o si el mensaje fue devuelto por mandatory (ninguna cola lo recibió).

Flores Herrera Oswaldo
Universidad Nacional Autónoma de México

6. La saga pública en lugar de llamar. En TransferenciaService sustituí NotificacionCliente por PublicadorDeEventos y borré las dos llamadas notificaciones.notificar(...) . Ahora la saga marca COMPLETADA , la guarda y después publica TransferenciaRealizada . El orden importa: si fuera al revés, la notificación podría avisar de una transferencia que la base todavía no dice qué pasó. En compensar , cuando queda COMPENSADA , publica TransferenciaRevertida ; si la devolución falla, no publica porque el dinero aún no regresó. El método avisar envuelve la publicación en un try/catch : si publicar falla, la transferencia no se revierte (el dinero ya se movió bien y el cliente recibe su 201), solo se pierde el aviso y queda en el log.

También actualicé TransferenciaServiceTest (cambio de NotificacionCliente por PublicadorDeEventos , verify(notificaciones).publicar(any()) y la prueba de la compensada, que ahora sí publica la revertida) para que ./mvnw verify quede en verde.

Prueba con notificación apagada. Reconstruí solo transferencia (docker compose up --build -d transferencia), apagué notificación y mandé otra transferencia de 50.

La transferencia terminó bien aunque la notificación estaba apagada. Pero un exchange no guarda mensajes: como todavía no había ninguna cola enlazada, el broker devolvió el mensaje por mandatory y ese aviso se perdió. Lo anoto porque se arregla en la sección 3, cuando la notificación declara su cola.

¿Qué garantiza el publisher confirm y qué no?

Garantiza que, cuando publicar() regresa sin excepción, el broker ya aceptó el mensaje (y que alguna cola lo recibió, gracias a mandatory). No garantiza que alguien lo haya leído ni procesado.

Notificación apagada: no afecta a la confirmación. El broker confirma cuando el mensaje queda en la cola durable; el aviso espera ahí hasta que la notificación vuelva. (En la prueba inicial no había cola, por eso se devolvió.)

Transferencia muere entre guardar(t) y avisar(...) : el confirm no ayuda, porque nunca se llegó a publicar. La base dice COMPLETADA , pero no hay evento, y nadie se entera. Es la doble escritura: base y broker son dos sistemas sin una transacción que los una.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 04:54:30 GMT

{"id":38,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50,"fecha":"2026-10-24T04:
54:29.963808325Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":900.00,"intentosDeCompensacion":0,"ac
tualizada":"2026-10-24T04:54:30.146380976Z","eventoId":"c4d78623-1540-45bf-a128-78fa8d976f09"}
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ docker compose logs transferencia | grep evento
transferencia-1  | 2026-10-24T04:54:30.167Z ERROR 1 --- [transferencia] [nio-8080-exec-4] m.m.transferencia.Transf
erenciaService   : Transferencia 38: quedo terminada pero el evento TransferenciaRealizada c4d78623-1540-45bf-a128
-78fa8d976f09 NO se publico: Ninguna cola recibio el evento c4d78623-1540-45bf-a128-78fa8d976f09 con la routing ke
y transferencia.realizada (NO_ROUTE)
```

Flores Herrera Oswaldo
Universidad Nacional Autónoma de México

3. Desacople en tiempo

Ahora el lado de notificacion:

- **Cola y binding.** ConfiguracionMensajeria declara el exchange transferencias (mismas propiedades que en transferencia, para que arranque en cualquier orden), la cola durable notificacion.transferencias y el binding con el patrón transferencia.*. La cola es de notificación y vive aunque la notificación esté apagada.
- **Evento local.** Notificación tiene su propia copia del record EventoTransferencia (mismos nombres de campo del JSON): cada servicio tiene la suya, igual que con HTTP.
- **Guardar avisos.** avisarDeTransferencia genera dos avisos por TransferenciaRealizada (TRANSFERENCIA_ENVIADA para origen y TRANSFERENCIA_RECIBIDA para destino) y uno por TransferenciaRevertida (DEVOLUCION_TRANSFERENCIA para origen). Es una sola transacción: se guardan todos o ninguno.
- **Consumidor con ack manual.** acknowledge-mode: manual y prefetch: 5. ConsumidorDeTransferencias lee el JSON, guarda los avisos y después llama basicAck.

Reconstruí la notificación (docker compose up --build -d notificacion) para que declarara su cola.

Experimento. Apagué la notificación, mandé tres transferencias (11, 12 y 13) y revisé la cola: tenía 3 Ready y 0 Unacked. Al encender la notificación, la cola bajó a 0 y el log muestra una línea por evento (redelivered=false, 2 aviso(s) nuevo(s)). La bandeja de Beto quedó con TRANSFERENCIA_RECIBIDA por 11, 12 y 13.

Esto es el desacople en tiempo: productor y consumidor ya no tienen que estar vivos al mismo tiempo; el recado espera en la cola durable.

Transferencia revertida. Levanté transferencia con FALLAR_ABONO_A_CLABE=002180000000000002 para que el abono a Beto falle a propósito, y mandé 150 de Ana a Beto. Respondió 502 con estado COMPENSACION_PENDIENTE: el abono falló como esperaba, pero la devolución a Ana también falló en ese momento, así que el dinero quedó pendiente de regresar. **[COMPLETAR después de comprobar: El reintento automático de la saga terminó la devolución (estado COMPENSADA, saldo de Ana de vuelta) y entonces llegó a su bandeja el aviso DEVOLUCION_TRANSFERENCIA por 150.]**

Hasta la S08 nadie le avisaba al cliente que su transferencia se había revertido, con el evento TransferenciaRevertida ahora sí. Después volví a levantar transferencia sin la variable para que el abono funcionara normal.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ for m in 11 12 13; do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"002180000000000001\",\"claveDestino\":\"002180000000000002\",\"monto\":$m}"
done
201
201
201

oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     3       0
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":59,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","f
echa":"2026-10-24T05:14:11.035487z"},{"id":57,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA
 por 12, saldo resultante 623.00","fecha":"2026-10-24T05:14:11.029847z"},{"id":55,"clabeCuenta":"00218000000000000
2","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-24T05:14:10.930572z"}]
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":150}'
HTTP/1.1 502 Bad Gateway
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sat, 24 Oct 2026 05:17:54 GMT

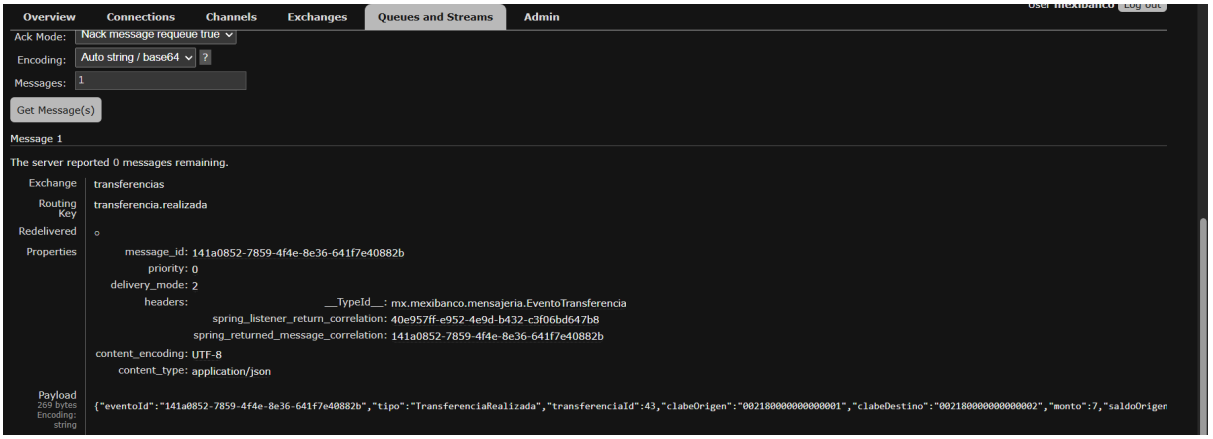
{"type":"about:blank","title":"Transferencia no realizada","status":502,"detail":"No se pudo abonar a la CLABE 002
180000000000002 y la devolucion de 150 a la CLABE 002180000000000001 tampoco pudo hacerse todavia. El dinero salio
 de origen; el banco reintentara la devolucion automaticamente.","instance":"/transferencias","transferencia":42,"
estado":"COMPENSACION_PENDIENTE","motivo":"El abono a la CLABE 002180000000000002 fallo: Falla provocada con FALLA
R_ABONO_A_CLABE=002180000000000002"}
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
[{"id":58,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00","fe
cha":"2026-10-24T05:14:11.035309z"},{"id":56,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA p
or 12, saldo resultante 877.00","fecha":"2026-10-24T05:14:11.029782z"},{"id":54,"clabeCuenta":"002180000000000001"
,"mensaje":"TRANSFERENCIA_ENVIADA por 11, saldo resultante 889.00","fecha":"2026-10-24T05:14:10.930421z"}]
```

4. **El duplicado.**

La mensajería se entrega al menos una vez (at-least-once): un mismo evento puede llegar dos veces. Mi consumidor original, si recibía dos veces el mismo evento, guardaba dos juegos de avisos.

Cómo lo provoqué. Desde el panel: con notificación apagada hice una transferencia (1 Ready), en Get messages usé Nack message requeue true para copiar el payload sin consumirlo, y lo publiqué otra vez en el exchange transferencias con la routing key transferencia.realizada y el mismo eventoId. Había 2 Ready; al encender la notificación, procesó ambos.

Flores Herrera Oswaldo
Universidad Nacional Autónoma de México



Antes del arreglo. La bandeja de Beto tiene dos veces TRANSFERENCIA_RECIBIDA por 7 (ids 61 y 63): el mismo evento guardó dos juegos de avisos.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":63,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00","fe
cha":"2026-10-24T05:29:12.854341Z"},{"id":61,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA
por 7, saldo resultante 643.00","fecha":"2026-10-24T05:29:12.720641Z"},{"id":59,"clabeCuenta":"002180000000000002"
,"mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-24T05:14:11.035487Z"},{"id":5
7,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2
026-10-24T05:14:11.029847Z"},{"id":55,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11,
 saldo resultante 611.00","fecha":"2026-10-24T05:14:10.930572Z"}]
```

El arreglo (idempotencia por eventoId). Creé una tabla eventos_procesados con evento_id como llave primaria. Al inicio de avisarDeTransferencia, en la misma transacción que saveAll(avisos), hago un INSERT … ON CONFLICT (evento_id) DO NOTHING y revisar cuántas filas insertó. Si fue 0, el evento ya se había procesado: regresó lista vacía (0 avisos nuevos) y el consumidor hace ack igual.

Por qué así:

Una sola sentencia atómica, no “consultar si existe y luego insertar”: entre esas dos llamadas se puede colar otra entrega del mismo mensaje.

No atrapo la violación de llave, porque en Postgres deja la transacción abortada.

Misma transacción que los avisos: si algo falla después de la marca, el rollback se lleva marca y avisos juntos, y la siguiente entrega empieza desde cero. Así no queda un evento marcado como procesado sin sus avisos.

Después del arreglo (repetí el mismo experimento, ahora con monto 8). Con la notificación apagada hice la transferencia y publiqué otra vez el mismo evento, así que la cola tenía 2 Ready. Al encender la notificación, el log muestra dos entregas del mismo eventoId: una con 2 aviso(s) nuevo(s) y la otra con 0 aviso(s) nuevo(s). La bandeja de Beto tiene un solo TRANSFERENCIA_RECIBIDA por 8.

Flores Herrera Oswaldo
Universidad Nacional Autónoma de México

5. La DLQ

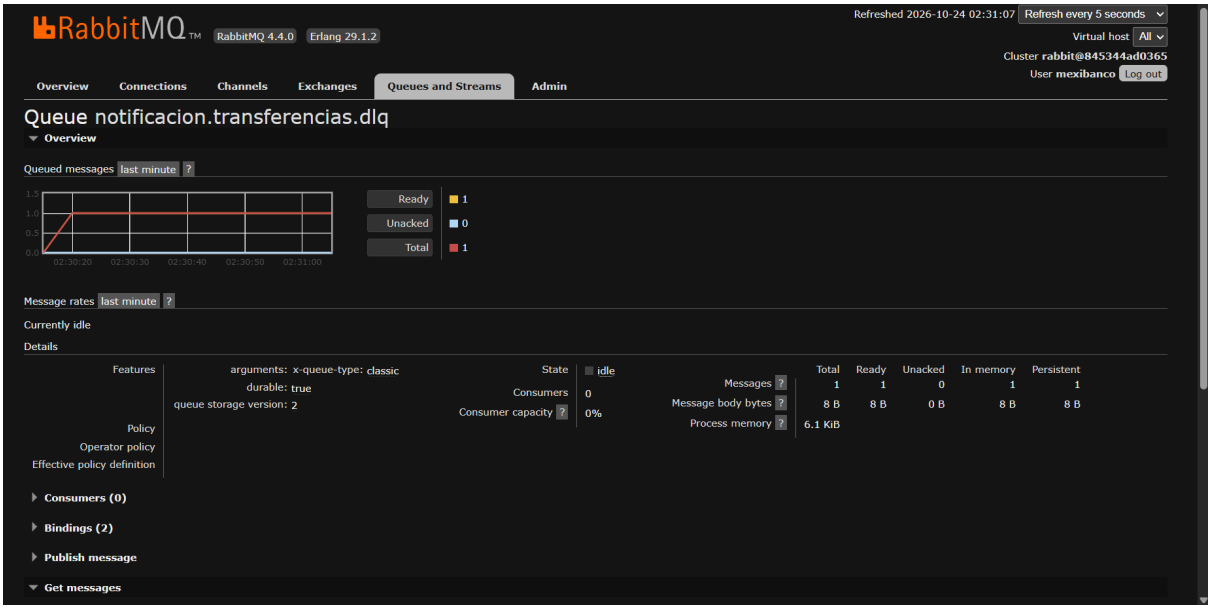
Un mensaje que nunca se va a poder procesar (poison pill) no se arregla reintentándolo; si se queda en la cola, estorba. Lo mandó a una dead letter queue para que una persona lo revisara.

Antes del arreglo. Desde el panel publiqué en el exchange transferencias, con routing key transferencia.realizada, el payload inválido {"hola mensaje malito :v":.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     2       0
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ docker compose logs notificacion | grep consumidor
notificacion-1  | 2026-10-24T05:58:07.202Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransf
erencias       : [consumidor] TransferenciaRealizada de la transferencia 44 (evento 69e130bb-4140-4fd3-9d9d-dac5c1
c8f8fa, redelivered=true): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-24T05:58:07.206Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransf
erencias       : [consumidor] TransferenciaRealizada de la transferencia 44 (evento 69e130bb-4140-4fd3-9d9d-dac5c1
c8f8fa, redelivered=false): 0 aviso(s) nuevo(s)

oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":65,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 8, saldo resultante 651.00","fe
cha":"2026-10-24T05:58:07.148049Z"},{"id":63,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA
por 7, saldo resultante 643.00","fecha":"2026-10-24T05:29:12.854341Z"},{"id":61,"clabeCuenta":"002180000000000002"
,"mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00","fecha":"2026-10-24T05:29:12.720641Z"},{"id":59
,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"20
26-10-24T05:14:11.035487Z"},{"id":57,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12,
saldo resultante 623.00","fecha":"2026-10-24T05:14:11.029847Z"},{"id":55,"clabeCuenta":"002180000000000002","mensa
je":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-24T05:14:10.930572Z"}]
```



Resultado. El consumidor falló al leer el JSON (JsonEOFException) y, como el ack es manual y nunca llegó al basicAck, el mensaje se quedó trabado en Unacked (0 Ready, 1 Unacked, 1 consumidor). No rebotó ni tiró el consumidor, y una transferencia normal posterior sí generó su aviso. Aun así, ese mensaje estorba: ocupa un lugar del prefetch y nadie lo va a poder procesar.

Flores Herrera Oswaldo
Universidad Nacional Autónoma de México

```
notificacion-1  | 2026-10-24T06:12:14.244Z  WARN 1 --- [notificacion] [ntContainer#0-1] s.a.r.l.ConditionalRejecti
ngErrorHandler : Execution of Rabbit message listener failed.
```

```
notificacion-1  | Caused by: com.fasterxml.jackson.core.io.JsonEOFException: Unexpected end-of-input within/betwee
n Object entries
```

```
notificacion-1  |       at mx.mexibanco.notificacion.mensajeria.ConsumidorDeTransferencias.recibir(ConsumidorDeTra
nsferencias.java:35) ~[!/:0.9.0]
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":67,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 5, saldo resultante 656.0
0","fecha":"2026-10-24T06:19:10.639928Z"},{"id":65,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENC
IA_RECIBIDA por 8, saldo resultante 651.00","fecha":"2026-10-24T05:58:07.148049Z"},{"id":63,"clabeCuenta":"0
02180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00","fecha":"2026-10-24T05:
29:12.854341Z"},{"id":61,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo r
esultante 643.00","fecha":"2026-10-24T05:29:12.720641Z"},{"id":59,"clabeCuenta":"002180000000000002","mensaj
e":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-24T05:14:11.035487Z"},{"id":57,
"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha
":"2026-10-24T05:14:11.029847Z"},{"id":55,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBI
DA por 11, saldo resultante 611.00","fecha":"2026-10-24T05:14:10.930572Z"}]
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":67,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 5, saldo resultante 656.0
0","fecha":"2026-10-24T06:19:10.639928Z"},{"id":65,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENC
IA_RECIBIDA por 8, saldo resultante 651.00","fecha":"2026-10-24T05:58:07.148049Z"},{"id":63,"clabeCuenta":"0
02180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00","fecha":"2026-10-24T05:
29:12.854341Z"},{"id":61,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo r
esultante 643.00","fecha":"2026-10-24T05:29:12.720641Z"},{"id":59,"clabeCuenta":"002180000000000002","mensaj
e":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-24T05:14:11.035487Z"},{"id":57,
"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha
":"2026-10-24T05:14:11.029847Z"},{"id":55,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBI
DA por 11, saldo resultante 611.00","fecha":"2026-10-24T05:14:10.930572Z"}]
```

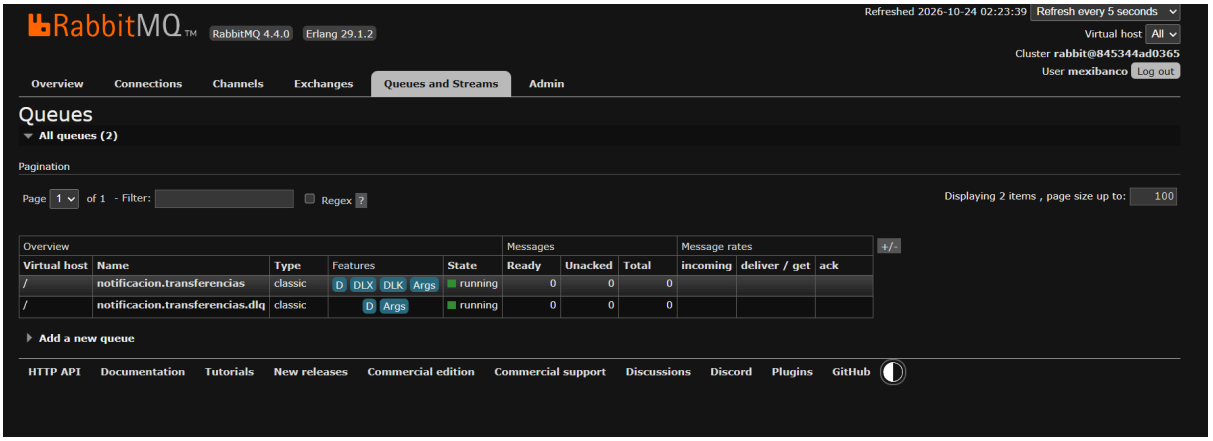
```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged consumers
notificacion.transferencias     0       1       1
```

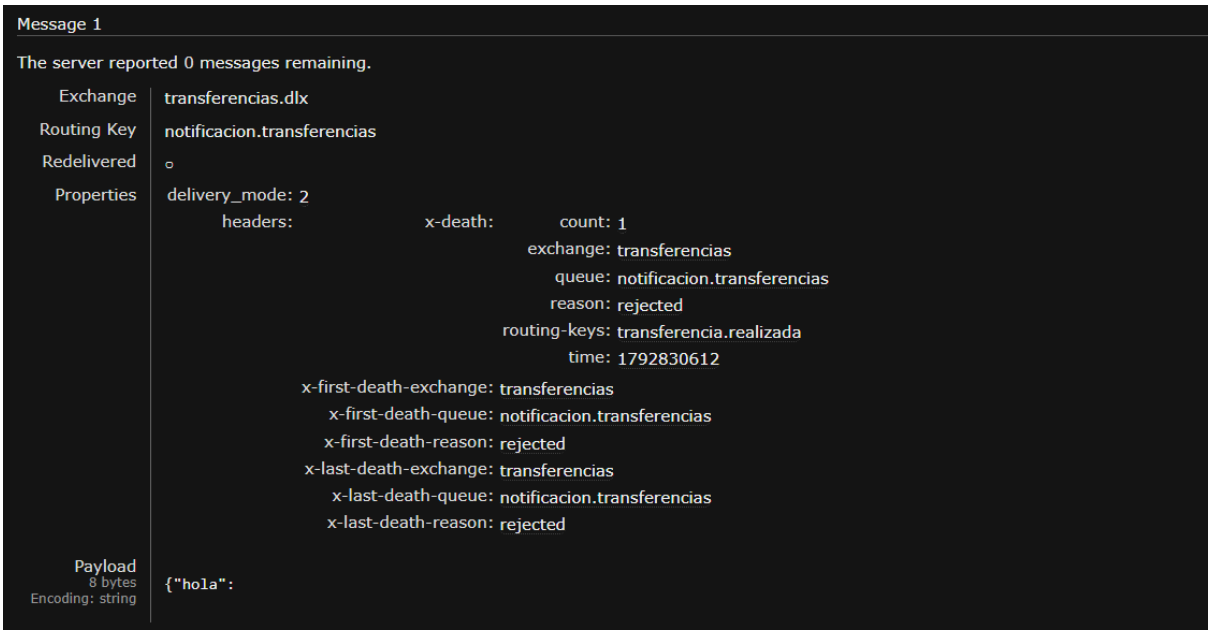
Configuración. La cola principal notificacion.transferencias ahora lleva x-dead-letter-exchange = transferencias.dlx y x-dead-letter-routing-key = notificacion.transferencias. Declaré el exchange transferencias.dlx (direct, durable) y la cola notificacion.transferencias.dlq (durable), enlazados con la clave notificacion.transferencias, en el mismo Declarables.

En el consumidor, si el JSON no se puede leer, hago canal.basicNack(etiqueta, false, false): el último false es requeue. Es el broker quien mueve el mensaje rechazado a la DLQ, no mi servicio. También mandó a la DLQ un evento sin eventold o tipo, o con un tipo desconocido; si falla el guardado (por ejemplo, la base), hago nack con requeue para reintentar.

Como la cola ya existía sin esos argumentos, RabbitMQ lanzó PRECONDITION_FAILED al arrancar. La borré desde el panel (con ella se fue el mensaje malo que estaba en Unacked) y reinicié la notificación con docker compose up --build -d notificacion. Ahora la cola principal muestra DLX y DLK en el panel.

Flores Herrera Oswaldo
Universidad Nacional Autónoma de México





Después del arreglo. Publiqué otro mensaje malo{"hola":) y el log del consumidor dijo “mensaje ilegible, va a la DLQ”. El mensaje apareció en notificacion.transferencias.dlq con la cabecera x-death (reason: rejected). Después hice una transferencia normal (monto 6) y siguió generando avisos.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/semana07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ docker compose logs notificacion | grep consumidor
notificacion-1  | 2026-10-24T08:30:12.413Z ERROR 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDe
Transferencias       : [consumidor] mensaje ilegible, va a la DLQ: Unexpected end-of-input within/between Ob
ject entries
```

¿Un JSON bien formado pero con un tipo que no existe, o sin eventold, también es veneno? Sí: aunque se pueda leer, tampoco se arregla reintentándolo, así que debe ir a la DLQ (nack sin reencolar), y así lo hace mi consumidor. En cambio, si lo que falló fue la base de datos, el mensaje está bien y conviene reencolarlo (requeue = true) para que se procese cuando la base vuelva.

Flores Herrera Oswaldo
Universidad Nacional Autónoma de México

6. Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA , ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?

El aviso se pierde. La saga guarda COMPLETADA en la base y luego intenta publicar; sin broker, publicar() lanza EventoNoPublicadoException (tras unos 2 s por el connection-timeout), el método avisar la atrapa y solo la deja en el log. El cliente recibe su 201 y la transferencia queda completada, pero el evento nunca sale, y al volver RabbitMQ nadie lo manda. El publisher confirm no lo arregla porque solo confirma mensajes que llegaron al broker; si no hay broker, no hay nada que confirmar ni reintentar.

El problema de fondo es la doble escritura: la base y el broker son dos sistemas sin una transacción que los una. La solución es el Transactional Outbox: guardar el evento en una tabla de la misma base y en la misma transacción que el cambio de estado, y que un relay lo publique después.

2. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?

Con el ack después de guardar, si la notificación muere entre guardar y el ack, el broker no recibió confirmación y vuelve a entregar el mensaje (redelivered=true): no se pierde nada, pero puede llegar dos veces (garantía at-least-once). Con el ack antes de guardar, si muere entre las dos líneas, el broker ya borró el mensaje el aviso se pierde, aunque nunca se repite (at-most-once). Para un banco prefiero repetir que perder, por eso el ack va después, y el costo de los duplicados lo cubre la idempotencia por eventoId de la sección 4.

3. Elige y di qué sacrificaste.

Elección: quitar la llamada síncrona a notificación y publicar un evento en RabbitMQ.
Porque: así el aviso no tumba una transferencia que ya movió el dinero, y notificación puede estar caída o lenta sin afectar al cliente.
Sacrifico: la certeza inmediata y simple. Ahora el aviso es eventual (puede tardar), puede duplicarse (necesito idempotencia), puede perderse si el broker está caído cuando la saga termina (a menos que implemente Outbox), y agregué piezas que operar y vigilar (broker, DLQ), además de perder la respuesta inmediata de "el aviso se envió".